

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA0508

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*
(BL2, BL3)

Activity Tool : Case Study (Medium)
Assessment Tool Weightage: 30%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.	BL2 – Understand	5
2	A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.	BL3 – Apply	5
3	Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.	BL3 – Apply	5
4	Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.	BL3 – Apply	5
5	Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.	BL2 – Understand	5
6	Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.	BL3 – Apply	5

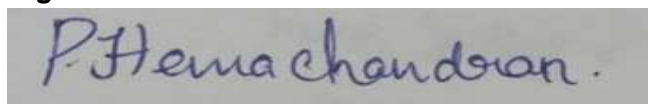
7	Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.	BL2 – Understand	5
8	A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.	BL3 – Apply	5
9	Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.	BL3 – Apply	5
10	Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.	BL3 – Apply	5

Code of Conduct:

I P. HEMACHANDRAN (192525440) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis	Good analysis with	Basic analysis	Weak or no analysis;

		with strong reasoning and insights	logical reasoning	with limited depth	lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

1: Online Shopping Platform — Normalize to 3NF

Given Relation

Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

The system stores information about orders, customers, and products.

Step 1: Identify Functional Dependencies

From the given attributes, the following functional dependencies can be identified:

1. **OrderID** → **CustID**
2. **CustID** → **CustName**
3. **ProdID** → **ProdName, Price**
4. **(OrderID, ProdID)** → **Qty**

Therefore, the combined dependency is:

(OrderID, ProdID) → CustID, CustName, ProdName, Price, Qty

Candidate Key

A single `OrderID` is not sufficient because one order can contain multiple products.

Therefore, the candidate key is:

(OrderID, ProdID)

Step 2: First Normal Form (1NF)

The relation is in **1NF** when:

- Each attribute contains atomic values.
- There are no repeating groups.

The given relation can be considered in 1NF because each field contains a single value.

Step 3: Second Normal Form (2NF)

For 2NF, there must be no **partial dependency** on part of the composite key.

The key is:

(OrderID, ProdID)

But:

- $\text{OrderID} \rightarrow \text{CustID}$
- $\text{CustID} \rightarrow \text{CustName}$
- $\text{ProdID} \rightarrow \text{ProdName, Price}$

Here, `CustID` and customer information depend only on `OrderID`, while product information depends only on `ProdID`.

Therefore, partial dependencies exist.

Decomposition into 2NF

Order Table

Order(OrderID, CustID)

Primary Key: `OrderID`

Customer Table

Customer(CustID, CustName)

Primary Key: `CustID`

Product Table

Product(ProdID, ProdName, Price)

Primary Key: `ProdID`

OrderItem Table

OrderItem(OrderID, ProdID, Qty)

Primary Key: (`OrderID`, `ProdID`)

Step 4: Third Normal Form (3NF)

For 3NF:

- The relation must be in 2NF.
- There must be no transitive dependency.

In the original relation:

OrderID → **CustID** → **CustName**

This is a **transitive dependency**.

Also:

ProdID → **ProdName, Price**

These dependencies have already been separated into their own relations.

Therefore, the final decomposition is in **3NF**.

Final 3NF Relations

Relation	Attributes	Primary Key
Customer	CustID, CustName	CustID
Product	ProdID, ProdName, Price	ProdID
Order	OrderID, CustID	OrderID
OrderItem	OrderID, ProdID, Qty	<u>(OrderID, ProdID)</u>

2: Hospital Database — Normalize to BCNF

Given Relation

Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Assume:

- **PID** uniquely identifies a patient.
- **DocID** uniquely identifies a doctor.
- **WardNo** uniquely identifies a ward.

Step 1: Identify Functional Dependencies

The important functional dependencies are:

1. **PID** → **PName, DocID, WardNo**

2. **DocID** → **DocName, Specialization**
3. **WardNo** → **WardName**

Therefore:

PID → **PName, DocID, DocName, Specialization, WardNo, WardName**

There are also transitive dependencies:

PID → **DocID** → **DocName, Specialization**

and

PID → **WardNo** → **WardName**

Step 2: Identify Candidate Key

Since **PID** determines all other attributes:

Candidate Key = PID

Step 3: Identify Anomalies

1. Update Anomaly

Suppose a doctor's specialization changes.

If the doctor treats many patients, the specialization must be updated in many rows. Missing one row creates inconsistent data.

Example:

PID	DocID	DocName	Specialization
------------	--------------	----------------	-----------------------

P101	D01	Dr. Kumar	Cardiology
------	-----	-----------	------------

P102	D01	Dr. Kumar	Cardiology
------	-----	-----------	------------

If Dr. Kumar changes specialization, both records must be updated.

2. Insertion Anomaly

A new doctor cannot be stored unless a patient is assigned to that doctor.

For example, if D05 is a newly appointed doctor, there is no way to store the doctor's details without a patient record.

3. Deletion Anomaly

If the only patient assigned to a doctor is deleted, the doctor's information may also be lost.

Similarly, deleting the only patient in a particular ward could remove the ward information.

Step 4: BCNF Condition

A relation is in **BCNF** when, for every non-trivial functional dependency:

$X \rightarrow Y$

X must be a **superkey**.

In the original relation:

DocID $\rightarrow$ DocName, Specialization

But DocID is not a superkey of the entire Patient relation.

Similarly:

WardNo $\rightarrow$ WardName

but WardNo is not a superkey of the entire relation.

Therefore, the original relation is **not in BCNF**.

Step 5: Decompose the Relation

Relation 1: Patient

Patient(PID, PName, DocID, WardNo)

Primary Key:

PID

Foreign Keys:

- $\text{DocID} \rightarrow \text{Doctor}$
- $\text{WardNo} \rightarrow \text{Ward}$

Relation 2: Doctor

Doctor(DocID, DocName, Specialization)

Primary Key:

DocID

Here:

$\text{DocID} \rightarrow \text{DocName, Specialization}$

Since `DocID` is the key, this relation satisfies BCNF.

Relation 3: Ward

Ward(WardNo, WardName)

Primary Key:

WardNo

Here:

$\text{WardNo} \rightarrow \text{WardName}$

Since `WardNo` is the key, this relation satisfies BCNF.

Relationship

$\text{Patient} \rightarrow \text{Doctor}$

`Patient.DocID` references `Doctor.DocID`

$\text{Patient} \rightarrow \text{Ward}$

Patient.WardNo references Ward.WardNo

Final BCNF Design

Relation	Attributes	Primary Key
Patient	PID, PName, DocID, WardNo	PID
Doctor	DocID, DocName, Specialization	DocID
Ward	WardNo, WardName	WardNo

3: University Enrollment — 2NF Decomposition

Given Relation

Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)

This relation stores student enrollment details, including student, course, faculty, and grade.

Step 1: Identify Functional Dependencies

From the relation, we can identify:

1. **SID → SName**
2. **CourseID → CourseName, Faculty**
3. **(SID, CourseID) → Grade**

Therefore:

(SID, CourseID) → SName, CourseName, Faculty, Grade

Step 2: Identify Candidate Key

A student can enroll in many courses, and a course can have many students.

Therefore, neither `SID` nor `CourseID` alone can uniquely identify an enrollment.

So the candidate key is:

(SID, CourseID)

Step 3: Check 1NF

The relation is in **1NF** because:

- Every attribute contains atomic values.
- There are no repeating groups.

Step 4: Check 2NF

A relation is in **2NF** when:

1. It is already in 1NF.
2. There is no partial dependency of a non-key attribute on part of a composite key.

The composite key is:

(SID, CourseID)

But we have:

SID → SName

SName depends only on SID, not on the whole composite key.

Also:

CourseID → CourseName, Faculty

These attributes depend only on CourseID, not on the whole composite key.

These are **partial dependencies**.

Therefore, the original relation is **not in 2NF**.

Step 5: Decomposition into 2NF

Separate the attributes according to their dependencies.

1. Student Relation

Student(SID, SName)

Primary Key:

SID

Functional dependency:

SID $\rightarrow$ SName

2. Course Relation

Course(CourseID, CourseName, Faculty)

Primary Key:

CourseID

Functional dependency:

CourseID $\rightarrow$ CourseName, Faculty

3. Enrollment Relation

Enrollment(SID, CourseID, Grade)

Primary Key:

(SID, CourseID)

Functional dependency:

(SID, CourseID) $\rightarrow$ Grade

Foreign Keys:

- **SID $\rightarrow$ Student(SID)**
- **CourseID $\rightarrow$ Course(CourseID)**

Final 2NF Decomposition

Relation	Attributes	Primary Key
Student	SID, SName	SID
Course	CourseID, CourseName, Faculty	CourseID
Enrollment	SID, CourseID, Grade	(SID, CourseID)

Why This Is 2NF

After decomposition:

- SName depends completely on SID.
- CourseName and Faculty depend completely on CourseID.
- Grade depends on the complete composite key (SID, CourseID).

Thus, **no partial dependency remains.**

4: Airline Booking — Normalize Up to BCNF

Given Relation

Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

We need to identify the functional dependencies and decompose the relation up to **BCNF**.

Step 1: Identify Functional Dependencies

Assume the following business rules:

1. **FlightNo** → **DeptCity, ArrCity**
2. **PassID** → **PassName**
3. **(FlightNo, Date, Seat)** → **PassID**

A flight operates between a fixed departure and arrival city:

FlightNo → **DeptCity, ArrCity**

A passenger ID identifies a passenger:

PassID → **PassName**

A particular seat on a particular flight/date identifies the passenger:

(FlightNo, Date, Seat) → PassID

Therefore:

(FlightNo, Date, Seat) → PassID, PassName, DeptCity, ArrCity

Step 2: Identify Candidate Key

The combination:

(FlightNo, Date, Seat)

uniquely identifies a booking.

Therefore:

Candidate Key = (FlightNo, Date, Seat)

Step 3: Check Functional Dependencies

The relation contains:

FlightNo → DeptCity, ArrCity

Here, `FlightNo` is only part of the candidate key, so this is a **partial dependency**.

Also:

PassID → PassName

`PassID` is not a superkey of the original relation.

Therefore, the original relation does **not satisfy BCNF**.

Step 4: BCNF Decomposition

Relation 1: Flight

Flight(FlightNo, DeptCity, ArrCity)

Primary Key:

FlightNo

Functional dependency:

FlightNo $\rightarrow$ DeptCity, ArrCity

Since `FlightNo` is the key, this relation is in **BCNF**.

Relation 2: Passenger

Passenger(PassID, PassName)

Primary Key:

PassID

Functional dependency:

PassID $\rightarrow$ PassName

Since `PassID` is the key, this relation is in **BCNF**.

Relation 3: Booking

Booking(FlightNo, Date, Seat, PassID)

Primary Key:

(FlightNo, Date, Seat)

Functional dependency:

(FlightNo, Date, Seat) $\rightarrow$ PassID

Since `(FlightNo, Date, Seat)` is the key, this relation is in **BCNF**.

Foreign Keys

- `Booking.FlightNo  $\rightarrow$  Flight.FlightNo`
- `Booking.PassID  $\rightarrow$  Passenger.PassID`

Final BCNF Schema

Relation	Attributes	Primary Key
Flight	FlightNo, DeptCity, ArrCity	FlightNo
Passenger	PassID, PassName	PassID
Booking	FlightNo, Date, Seat, PassID	(FlightNo, Date, Seat)

Step 5: Why It Is BCNF

For BCNF, every determinant must be a **superkey**.

Flight

FlightNo → **DeptCity, ArrCity**

FlightNo is the primary key.

Passenger

PassID → **PassName**

PassID is the primary key.

Booking

(FlightNo, Date, Seat) → **PassID**

(FlightNo, Date, Seat) is the primary key.

Therefore, all three relations satisfy **BCNF**.

5: HR System for a Multinational Company — Normalize to 3NF

Since no exact HR attributes were supplied, we can use a standard HR relation for the analysis:

Employee(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, JobID, JobTitle, Salary)

This represents employee, department, manager, and job information.

Step 1: Identify Functional Dependencies

The main functional dependencies are:

1. **EmpID $\rightarrow$ EmpName, DeptID, ManagerID, JobID, Salary**
2. **DeptID $\rightarrow$ DeptName, ManagerID**
3. **ManagerID $\rightarrow$ ManagerName**
4. **JobID $\rightarrow$ JobTitle**

Therefore, we have several dependencies where non-key attributes determine other non-key attributes.

Step 2: Identify Candidate Key

EmpID uniquely identifies an employee.

Therefore:

Candidate Key = EmpID

and

EmpID $\rightarrow$ all other attributes

Step 3: Identify Transitive Dependencies

A transitive dependency occurs when:

A $\rightarrow$ B and **B $\rightarrow$ C**, therefore **A $\rightarrow$ C**.

Dependency 1

EmpID $\rightarrow$ DeptID

and

DeptID $\rightarrow$ DeptName

Therefore:

EmpID $\rightarrow$ DeptName

This is a transitive dependency.

Dependency 2

EmpID $\rightarrow$ ManagerID

and

ManagerID $\rightarrow$ ManagerName

Therefore:

EmpID $\rightarrow$ ManagerName

This is also a transitive dependency.

Dependency 3

EmpID $\rightarrow$ JobID

and

JobID $\rightarrow$ JobTitle

Therefore:

EmpID $\rightarrow$ JobTitle

This is another transitive dependency.

Step 4: Problems in the Original Relation

The original HR relation has redundancy.

Update Anomaly

If a department changes its name, every employee belonging to that department must be updated.

For example:

EmpID DeptID DeptName

E101 D10 Finance

E102 D10 Finance

Changing Finance to another name requires multiple updates.

Insertion Anomaly

A new department cannot easily be inserted if it has no employees.

Deletion Anomaly

If the last employee in a department is deleted, the department information may also be lost.

Step 5: Convert to 3NF

To remove transitive dependencies, separate the entities.

1. Employee Table

Employee(EmpID, EmpName, DeptID, ManagerID, JobID, Salary)

Primary Key:

EmpID

2. Department Table

Department(DeptID, DeptName, ManagerID)

Primary Key:

DeptID

Functional dependency:

DeptID → DeptName, ManagerID

3. Manager Table

Manager(ManagerID, ManagerName)

Primary Key:

ManagerID

Functional dependency:

ManagerID $\rightarrow$ ManagerName

4. Job Table

Job(JobID, JobTitle)

Primary Key:

JobID

Functional dependency:

JobID $\rightarrow$ JobTitle

Final 3NF Design

Relation	Attributes	Primary Key
Employee	EmpID, EmpName, DeptID, ManagerID, JobID, Salary	EmpID
Department	DeptID, DeptName, ManagerID	DeptID
Manager	ManagerID, ManagerName	ManagerID
Job	JobID, JobTitle	JobID

Foreign Keys

- Employee.DeptID $\rightarrow$ Department.DeptID
- Employee.ManagerID $\rightarrow$ Manager.ManagerID
- Employee.JobID $\rightarrow$ Job.JobID
- Department.ManagerID $\rightarrow$ Manager.ManagerID

Step 6: 3NF Verification

A relation is in **3NF** when, for every functional dependency:

X $\rightarrow$ A

either:

- X is a superkey, or
- A is a prime attribute.

In our decomposed relations, the determinants are keys:

EmpID → Employee details

DeptID → DeptName, ManagerID

ManagerID → ManagerName

JobID → JobTitle

Therefore, the relations satisfy 3NF.

Final Conclusion

The original HR schema contains several **transitive dependencies**, particularly:

EmpID → DeptID → DeptName

EmpID → ManagerID → ManagerName

EmpID → JobID → JobTitle

6: Library Database — MVDs and 4NF Decomposition

Given Relation

Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

The library stores information about members and the books they borrow.

Step 1: Identify Functional Dependencies

We can identify the following functional dependencies:

1. **MemberID** → MemberName
2. **BookID** → Title, Author, Genre

The borrowing date depends on the member-book combination:

3. $(\text{MemberID}, \text{BookID}) \rightarrow \text{BorrowDate}$

Step 2: Identify Multivalued Dependencies (MVDs)

A multivalued dependency exists when one attribute determines a set of independent values of another attribute.

For this case, consider a member who can borrow multiple books:

$\text{MemberID} \twoheadrightarrow \text{BookID}$

A member can also have multiple borrowing dates independently associated with books:

$\text{MemberID} \twoheadrightarrow \text{BorrowDate}$

These independent multivalued facts can cause unnecessary combinations and redundancy.

Example

Suppose member M101 borrows:

- Book B01
- Book B02

and has borrowing dates:

- 10-08-2026
- 12-08-2026

The unnormalized representation can generate combinations between books and dates, even when those combinations do not represent separate borrowing events.

Step 3: Why the Relation Is Not in 4NF

A relation is in **4NF** if, for every non-trivial multivalued dependency:

$X \twoheadrightarrow Y$

X must be a **superkey**.

Here:

$\text{MemberID} \twoheadrightarrow \text{BookID}$

But `MemberID` alone is not a superkey of the original relation.

Therefore, the relation violates **4NF**.

Step 4: Decompose the Relation

We separate independent multivalued facts.

1. Member Relation

Member(MemberID, MemberName)

Primary Key:

MemberID

Functional dependency:

MemberID → **MemberName**

2. Book Relation

Book(BookID, Title, Author, Genre)

Primary Key:

BookID

Functional dependency:

BookID → **Title, Author, Genre**

3. Borrowing Relation

Borrowing(MemberID, BookID, BorrowDate)

Primary Key:

(MemberID, BookID, BorrowDate)

This relation records the actual borrowing event.

Foreign Keys:

- $\text{MemberID} \rightarrow \text{Member}(\text{MemberID})$
- $\text{BookID} \rightarrow \text{Book}(\text{BookID})$

Final 4NF Schema

Relation	Attributes	Primary Key
Member	MemberID, MemberName	MemberID
Book	BookID, Title, Author, Genre	BookID
Borrowing	MemberID, BookID, BorrowDate (MemberID, BookID, BorrowDate)	

Step 5: 4NF Verification

Member

MemberID $\rightarrow$ **MemberName**

MemberID is the key.

Book

BookID $\rightarrow$ **Title, Author, Genre**

BookID is the key.

Borrowing

The borrowing information is represented as a separate relation, so the independent multivalued facts causing the 4NF violation are removed. ✓

7: Telecom Billing System — Keys and Functional Dependencies

Since the exact telecom schema is not provided, consider a standard telecom billing relation:

Telecom(CustomerID, CustomerName, PhoneNo, PlanID, PlanName, MonthlyCharge, BillID, BillDate, Usage, TotalAmount)

This relation contains customer, mobile plan, and billing information.

Step 1: Identify Functional Dependencies

The important functional dependencies are:

1. **CustomerID** → **CustomerName, PhoneNo, PlanID**
2. **PhoneNo** → **CustomerID**
3. **PlanID** → **PlanName, MonthlyCharge**
4. **BillID** → **CustomerID, BillDate, Usage, TotalAmount**

Therefore:

CustomerID → **CustomerName, PhoneNo, PlanID**

PlanID → **PlanName, MonthlyCharge**

BillID → **CustomerID, BillDate, Usage, TotalAmount**

Step 2: Identify Candidate Keys

A **candidate key** is a minimal attribute or set of attributes that uniquely identifies each tuple.

Candidate Key 1: CustomerID

CustomerID uniquely identifies a customer.

Therefore:

CustomerID → **all customer-specific attributes**

So, **CustomerID** is a candidate key for the customer information.

Candidate Key 2: PhoneNo

Assuming every phone number belongs to exactly one customer:

PhoneNo → **CustomerID**

Therefore, **PhoneNo** can also uniquely identify a customer.

So:

PhoneNo is another candidate key for customer information.

Candidate Key 3: BillID

Each bill has a unique billing number.

Therefore:

BillID is a candidate key for billing records.

Step 3: Primary Key

A primary key is the candidate key selected to identify records.

For the complete billing relation, we can select:

Primary Key = BillID

Other candidate keys, where applicable, become alternate keys.

Step 4: Transitive Dependencies

There are also transitive dependencies.

For example:

CustomerID $\rightarrow$ PlanID

and

PlanID $\rightarrow$ PlanName, MonthlyCharge

Therefore:

CustomerID $\rightarrow$ PlanName, MonthlyCharge

This is a **transitive dependency**.

Similarly:

BillID $\rightarrow$ CustomerID $\rightarrow$ PlanID

Therefore:

BillID $\rightarrow$ PlanID

and consequently:

BillID → **PlanName, MonthlyCharge**

Step 5: Summary of Keys

Attribute	Role
CustomerID	Candidate Key
PhoneNo	Candidate Key
BillID	Candidate Key
BillID	Selected Primary Key
PlanID	Determinant, but not a key of the complete relation

Step 6: Summary of Functional Dependencies

Functional Dependency	Meaning
CustomerID → CustomerName, PhoneNo, PlanID	CustomerID identifies customer details
PhoneNo → CustomerID	Phone number identifies customer
PlanID → PlanName, MonthlyCharge	PlanID identifies plan details
BillID → CustomerID, BillDate, Usage, TotalAmount	BillID identifies bill details

Important Observation

The original telecom relation contains repeated plan and customer information.

For example:

BillID	CustomerID	CustomerName	PlanID	PlanName
B001	C101	Arun	P10	Premium
B002	C101	Arun	P10	Premium
B003	C101	Arun	P10	Premium

The same customer and plan details are repeated across multiple bills.

This can cause:

Update anomaly: Changing the plan name requires multiple updates.

Insertion anomaly: A new plan may not be stored without a customer.

Deletion anomaly: Deleting the last bill may remove useful customer or plan information.

8: Retail Inventory System — Complete Normalization

Given Relation

Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

We need to identify the functional dependencies and normalize the relation completely.

Step 1: Identify Functional Dependencies

From the given schema, assume:

1. **PID $\rightarrow$ PName, SupplierID, Category, Price, Warehouse**
2. **SupplierID $\rightarrow$ SupplierName**

Therefore, there is a transitive dependency:

PID $\rightarrow$ SupplierID $\rightarrow$ SupplierName

Step 2: Identify Candidate Key

PID uniquely identifies each product.

Therefore:

Candidate Key = PID

and:

PID $\rightarrow$ PName, SupplierID, Category, Price, Warehouse

Step 3: Check 1NF

The relation is in **1NF** because each attribute contains atomic values and there are no repeating groups.

Step 4: Check 2NF

The primary key is a single attribute:

PID

Since there is no composite key, partial dependency cannot occur.

Therefore, the relation satisfies **2NF**.

Step 5: Check 3NF

We have:

PID → SupplierID

and

SupplierID → SupplierName

Therefore:

PID → SupplierName

This is a **transitive dependency**.

So the relation is **not in 3NF**.

Step 6: Decompose into 3NF

Product Table

Product(PID, PName, SupplierID, Category, Price, Warehouse)

Primary Key:

PID

Supplier Table

Supplier(SupplierID, SupplierName)

Primary Key:

SupplierID

Functional dependency:

SupplierID $\rightarrow$ SupplierName

Step 7: Check BCNF

Product Relation

PID $\rightarrow$ PName, SupplierID, Category, Price, Warehouse

PID is the candidate key.

Therefore, the determinant is a superkey.

Supplier Relation

SupplierID $\rightarrow$ SupplierName

SupplierID is the candidate key.

Therefore, the determinant is a superkey.

Thus, both relations satisfy **BCNF** under these assumptions.

Final Normalized Design

Relation	Attributes	Primary Key
Product	PID, PName, SupplierID, Category, Price, Warehouse	PID
Supplier	SupplierID, SupplierName	SupplierID

Foreign Key

`Product.SupplierID` references `Supplier.SupplierID`.

Example

Before Normalization

PID	PName	SupplierID	SupplierName	Category	Price	Warehouse
P01	Laptop	S01	ABC Traders	Electronics	50000	W1
P02	Mouse	S01	ABC Traders	Electronics	800	W1
P03	Keyboard	S02	XYZ Suppliers	Electronics	1500	W2

`ABC Traders` is repeated for several products.

After Normalization

Supplier

SupplierID	SupplierName
S01	ABC Traders
S02	XYZ Suppliers

Product

PID	PName	SupplierID	Category	Price	Warehouse
P01	Laptop	S01	Electronics	50000	W1
P02	Mouse	S01	Electronics	800	W1
P03	Keyboard	S02	Electronics	1500	W2

Anomalies Removed

Update anomaly: Supplier details need to be changed only once in the Supplier table.

Insertion anomaly: A new supplier can be added even before any product is entered.

Deletion anomaly: Deleting a product does not remove the supplier's information.

9: School ERP — Fully Normalized Design up to BCNF

Since the question does not provide an exact School ERP schema, we can use a typical poorly normalized relation:

SchoolERP(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Grade)

This relation stores student, class, teacher, subject, and result information together.

Step 1: Identify Functional Dependencies

The main functional dependencies are:

1. **StudentID $\rightarrow$ StudentName, ClassID**
2. **ClassID $\rightarrow$ ClassName, TeacherID**
3. **TeacherID $\rightarrow$ TeacherName**
4. **SubjectID $\rightarrow$ SubjectName**
5. **(StudentID, SubjectID) $\rightarrow$ Grade**

Therefore, we also have transitive dependencies:

StudentID $\rightarrow$ ClassID $\rightarrow$ ClassName

StudentID $\rightarrow$ ClassID $\rightarrow$ TeacherID $\rightarrow$ TeacherName

Step 2: Identify Candidate Key

A student can study several subjects, so `StudentID` alone does not identify a grade record.

The combination:

(StudentID, SubjectID)

uniquely identifies the student's grade for a subject.

Therefore:

Candidate Key = (StudentID, SubjectID)

Step 3: Problems in the Original Schema

1. Update Anomaly

If a teacher's name changes, every student record associated with that teacher must be updated.

2. Insertion Anomaly

A new subject or teacher cannot easily be inserted without creating a student-related record.

3. Deletion Anomaly

Deleting the last student taking a subject could unintentionally remove information about that subject.

4. Data Redundancy

Student, class, teacher, and subject information is repeated many times.

Step 4: First Normal Form (1NF)

The relation is in **1NF** because:

- Values are atomic.
- There are no repeating groups.
- Each record contains a single value for each attribute.

Step 5: Second Normal Form (2NF)

The key is:

(StudentID, SubjectID)

But:

StudentID → StudentName, ClassID

and:

SubjectID $\rightarrow$ SubjectName

These attributes depend only on part of the composite key.

Therefore, the original relation is **not in 2NF**.

Remove Partial Dependencies

Create separate relations:

Student(StudentID, StudentName, ClassID)

Subject(SubjectID, SubjectName)

Result(StudentID, SubjectID, Grade)

Step 6: Third Normal Form (3NF)

There are still transitive dependencies:

StudentID $\rightarrow$ ClassID

and

ClassID $\rightarrow$ ClassName, TeacherID

Therefore:

StudentID $\rightarrow$ ClassName

and:

StudentID $\rightarrow$ TeacherID

Also:

TeacherID $\rightarrow$ TeacherName

These must be separated.

Step 7: Final Decomposition

1. Student

Student(StudentID, StudentName, ClassID)

Primary Key:

StudentID

Foreign Key:

ClassID → Class

2. Class

Class(ClassID, ClassName, TeacherID)

Primary Key:

ClassID

Foreign Key:

TeacherID → Teacher

3. Teacher

Teacher(TeacherID, TeacherName)

Primary Key:

TeacherID

4. Subject

Subject(SubjectID, SubjectName)

Primary Key:

SubjectID

5. Result

Result(StudentID, SubjectID, Grade)

Primary Key:

(StudentID, SubjectID)

Foreign Keys:

- StudentID → Student
- SubjectID → Subject

Final BCNF Design

Relation	Attributes	Primary Key
Student	StudentID, StudentName, ClassID	StudentID
Class	ClassID, ClassName, TeacherID	ClassID
Teacher	TeacherID, TeacherName	TeacherID
Subject	SubjectID, SubjectName	SubjectID
Result	StudentID, SubjectID, Grade	(StudentID, SubjectID)

Step 8: BCNF Verification

A relation is in **BCNF** if every determinant is a superkey.

Student

StudentID → StudentName, ClassID

StudentID is the key.

Class

ClassID → ClassName, TeacherID

ClassID is the key.

Teacher

TeacherID → **TeacherName**

TeacherID is the key.

Subject

SubjectID → **SubjectName**

SubjectID is the key.

Result

(StudentID, SubjectID) → **Grade**

(StudentID, SubjectID) is the key.

Therefore, all relations satisfy **BCNF**.

Simple Structure

Student

↓ belongs to

Class

↓ handled by

Teacher

and

Student + Subject

↓

Result / Grade

10: Movie Streaming Platform — 5NF Decomposition

Given Relation

Since no exact schema is provided, consider a typical movie-streaming relation:

Streaming(MovieID, MovieTitle, ActorID, ActorName, LanguageID, Language, PlatformID, PlatformName)

A movie may have multiple actors, be available in multiple languages, and be available on multiple streaming platforms.

Step 1: Identify the Join Dependency

There are three independent relationships:

1. **Movie** ↔ **Actor**
2. **Movie** ↔ **Language**
3. **Movie** ↔ **Platform**

For example:

- Movie **M01** has actors **A01** and **A02**.
- Movie **M01** is available in languages **English** and **Tamil**.
- Movie **M01** is available on platforms **P01** and **P02**.

If all three relationships are stored in one table, unnecessary combinations can be generated.

This gives a **join dependency**:

(MovieID, ActorID), (MovieID, LanguageID), (MovieID, PlatformID)

That means the original relation can be reconstructed by joining these three smaller relations.

Step 2: Why 5NF Is Required

A relation is in **5NF (Fifth Normal Form)** when every non-trivial join dependency is implied by the candidate keys.

The original relation contains independent many-to-many relationships. Keeping all of them together can create redundancy.

For example:

MovieID	ActorID	LanguageID	PlatformID
M01	A01	L01	P01
M01	A01	L01	P02
M01	A01	L02	P01
M01	A01	L02	P02
M01	A02	L01	P01
M01	A02	L01	P02
M01	A02	L02	P01
M01	A02	L02	P02

Many combinations are produced even though the three relationships are independent.

Step 3: Decompose the Relation

We separate the independent relationships.

1. Movie-Actor Relation

MovieActor(MovieID, ActorID)

Primary Key:

(MovieID, ActorID)

2. Movie-Language Relation

MovieLanguage(MovieID, LanguageID)

Primary Key:

(MovieID, LanguageID)

3. Movie-Platform Relation

MoviePlatform(MovieID, PlatformID)

Primary Key:

(MovieID, PlatformID)

4. Movie Relation

Movie(MovieID, MovieTitle)

Primary Key:

MovieID

5. Actor Relation

Actor(ActorID, ActorName)

Primary Key:

ActorID

6. Language Relation

Language(LanguageID, Language)

Primary Key:

LanguageID

7. Platform Relation

Platform(PlatformID, PlatformName)

Primary Key:

PlatformID

Step 4: Final 5NF Design

Relation	Attributes	Primary Key
Movie	MovieID, MovieTitle	MovieID
Actor	ActorID, ActorName	ActorID
Language	LanguageID, Language	LanguageID
Platform	PlatformID, PlatformName	PlatformID
MovieActor	MovieID, ActorID	(MovieID, ActorID)
MovieLanguage	MovieID, LanguageID	(MovieID, LanguageID)
MoviePlatform	MovieID, PlatformID	(MovieID, PlatformID)

Step 5: Lossless-Join Verification

The original information can be reconstructed using joins:

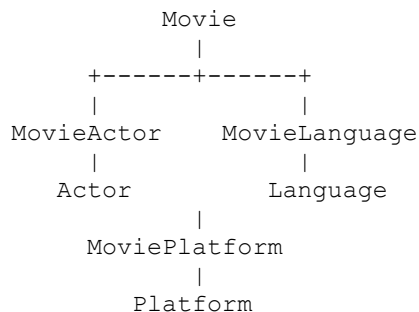
MovieActor ⋈ **MovieLanguage** ⋈ **MoviePlatform**

along with the descriptive tables:

Movie, Actor, Language, Platform

Thus, the decomposition does not lose the relationships represented by the original schema, assuming the three many-to-many relationships are genuinely independent.

Join representation



Step 6: Why the Decomposition Is in 5NF

The original relation has a non-trivial join dependency involving:

MovieActor, MovieLanguage, MoviePlatform

These independent relationships are separated into their own relations.

